



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

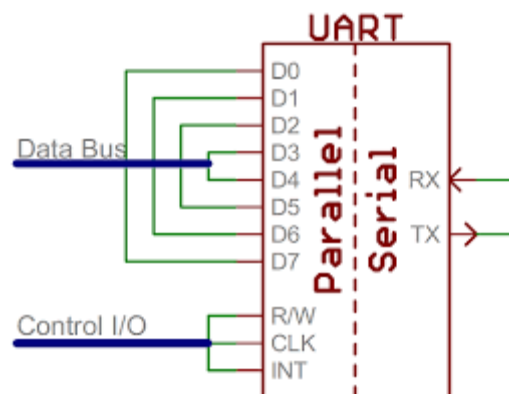
Grado en Informática Industrial y Robótica

Asignatura: Redes Industriales

Código de la asignatura: 14541

Práctica de Laboratorio

“Práctica 1. Comunicación Serie utilizando la
UART”



Profesor responsable:

Dr. Lenin G. Lemus Zúñiga

Alumnos Implicados:

Mario Martínez Carneros

Francisco Nortes Novikov

Página en Blanco

Contenido

Introducción:.....	7
Funcionamiento.....	7
Características Principales de la UART:.....	8
Control de flujo:.....	8
Objetivo:.....	10
Material para el desarrollo de la práctica:.....	10
Desarrollo de la práctica:.....	13
Primera parte: Crear un programa que envíe datos a través de uno de los puertos COM.....	13
Segunda parte: Visualización de las señales al transmitir el BYTE correspondiente al carácter 'A'	15
Ampliaciones.....	17
Apéndice A.....	20
Referencias Bibliográficas:.....	21

Página en blanco

Tabla de figuras

Figura 1.Comunicación entre UARTs.....7

Figura 2. Señales para el control de flujo.....8

Figura 3: Visualización de la letra A enviada por UART en el osciloscopio.....15

Figura 4: Conexión de la ESP32 mediante UART al cable USB TTL del ordenador y al ordenador mediante USB..... 18

Figura 5: Comunicación ESP32 <-> Ordenador..... 19

Página en blanco

Introducción:

La comunicación serie es un protocolo de transmisión de datos que envía información secuencialmente, bit a bit, a través de un solo cable o canal, a diferencia de la paralela que usa múltiples vías. Es fundamental en electrónica digital para conectar microcontroladores, sensores y computadoras, destacando por su simplicidad, uso de pocos pines y largas distancias.

El protocolo UART (Universal Asynchronous Receiver-Transmitter) es un protocolo de comunicación serie asíncrono, ampliamente utilizado para la comunicación entre microcontroladores, sensores, módulos Bluetooth y computadoras. Utiliza solo dos cables (TX y RX) para la transmisión bidireccional de datos bit a bit, sin requerir una señal de reloj compartida. Los dispositivos deben configurarse a la misma velocidad en baudios.

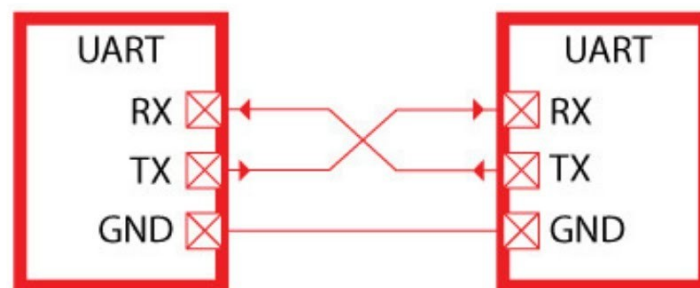


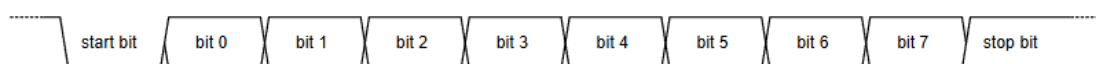
Figura 1. Comunicación entre UARTs.

Funcionamiento

El protocolo UART es un protocolo de comunicación de hardware P2P (peer-to-peer) en donde uno de los extremos puede ser un MCU (microcontrolador) y el otro extremo puede ser otro MCU, un sensor o un ordenador personal (a través de un convertidor USB a UART).

Atención: Una UART envía la información por BYTES, un BYTE en cada transmisión, y dado que la transmisión es asíncrona cuando una de las UART va a enviar el BYTE, utiliza un formato conocido como formato de BIT.

La transferencia de datos se realiza en serie. Comienza con un bit de inicio, generalmente activando la lógica a nivel bajo durante un ciclo de reloj. En los siguientes n ciclos de reloj, se envían n bits secuencialmente desde el transmisor (n puede ser 6, 7 o 8 en general suele ser 8). Opcionalmente, se puede añadir un bit de paridad para mejorar la fiabilidad de la transferencia. Finalmente, el cable de datos suele activarse para indicar el final de la transferencia. El bit de parada puede tener una duración de $\frac{1}{2}$ tiempo de bit, 1 tiempo de bit o dos tiempos de bit.



Atención: Las comunicaciones a través de la UART se especifican indicando la tasa de transmisión (9600 bits/segundo), el número de bits a transmitir (6, 7 y 8) el número de bits de parada y si utiliza un bit de paridad para determinar si hay errores en la transmisión. De forma que las características de la transmisión se especifican indicando la velocidad de transmisión

en bits/seg, un dígito que indica el número de bits, una letra para indicar la paridad (N=Sin paridad, E=paridad par ó O= paridad impar) y finalmente el número de bits de parada. Por ejemplo: 9600 8N1

Características Principales de la UART:

- Comunicación Asíncrona: No utiliza una señal de reloj común para sincronizar transmisor y receptor. La sincronización se logra mediante bits de inicio (start) y parada (stop) añadidos a cada trama de datos.
- Conexiones: Utiliza principalmente dos pines: TX (Transmisión) y RX (Recepción).
- Comunicación: Full-Duplex (ambos extremos pueden transmitir simultáneamente).
- Conexión entre dispositivos: El pin TX de un dispositivo se conecta al RX del otro, y viceversa, compartiendo la misma tierra (GND).
- Configuración: La velocidad de transmisión, conocida como baudios (ej. 9600, 115200), debe ser idéntica en ambos dispositivos.
- Niveles de Tensión: UART comúnmente opera a niveles TTL (0V - 3.3V o 5V), diferenciándose de RS-232, que utiliza voltajes más altos.
- Es Importante conectar las señales de masa.

Control de flujo:

Para que dos dispositivos puedan comunicarse es fundamental utilizar un protocolo, en este caso el Protocolo UART.

Para ello es necesario agregar las señales CTS (Clear To Send) y RTS (Ready To Send)

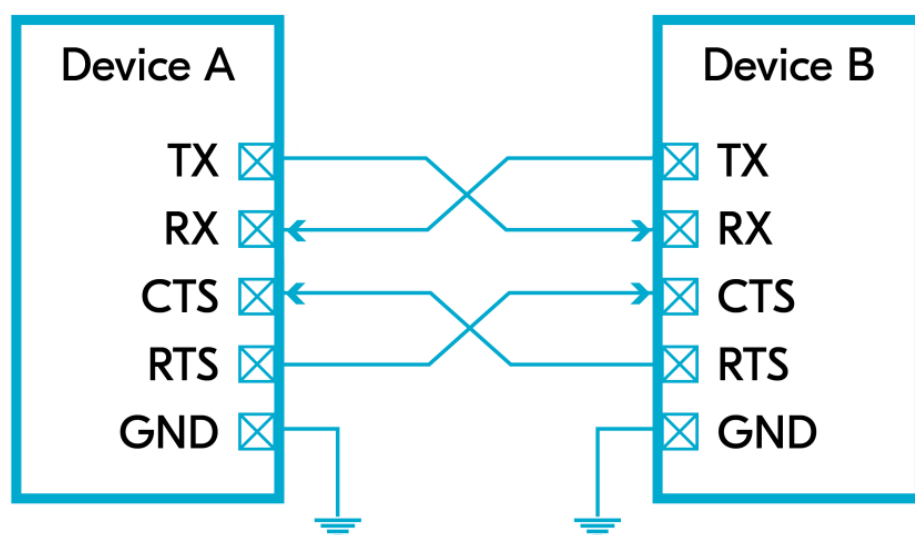
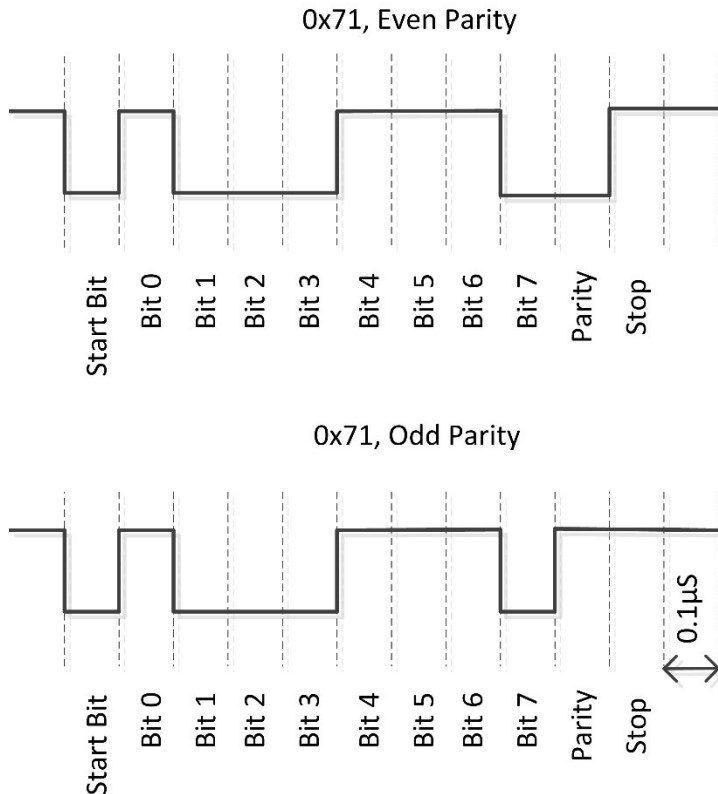


Figura 2. Señales para el control de flujo.

Por ejemplo, cuando la UART "Device A" está preparada para enviar información, activa la señal CTS, la UART "Device B" detecta que en la entrada RTS se ha activado, lo que significa que van a empezar a llegar los BYTES que contienen la información. Y acto seguido la UART

“Device A” inicia la transmisión de la información BYTE por BYTE. Cuando la UART “Device B” ha terminado de transmitir todos los datos de la información desactiva la señal CTS.

A continuación, se muestra el formato de cada BYTE transmitido, para una comunicación con paridad PAR



Pregunta: si el tiempo de bit es de 0.1 micro segundo. ¿Cuál es la tasa de transmisión?

Respuesta: 10000000 Bits/seg

¿Cómo se especifica cada una de estas transmisiones?

Respuesta (paridad par): 8E1

Respuesta (paridad impar): 8O1

Escriba en formato binario, los bits que se envían. Respuesta: 01110001₂

Escriba en formato hexadecimal los bits que pasan a través del canal de comunicación:

Respuesta: 0x81

Objetivo:

- Aprender a enviar y recibir datos a través de UART en modo asíncrono
- Aprender a configurar el hardware periférico UART mediante la API de UART.
- Examinar y practicar el uso de la API del controlador UART, para enviar datos de un PC a un microcontrolador

Material para el desarrollo de la práctica:

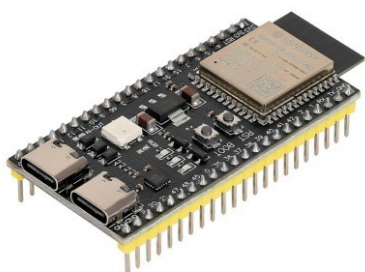
Hardware:

1 Ordenador Personal



1 Cable USB a TTL

1 Placa de desarrollo ESP32-S3



Software:

En el ordenador personal debe tener instalados los siguientes entornos de desarrollo:

- Arduino IDE
- Netbeans
- Driver para el cable USB a TTL

Interconexión de los componentes:

En el ESP32-s3

MCU	UART	RX GPIO	TX GPIO	Note
ESP32	UART0	3	1	Serial
	UART1	9	10	Serial1
	UART2	16	17	Serial2
ESP32-S2	UART0	44	43	Serial or Serial0 (*)
	UART1	18	17	Serial1
ESP32-S3	UART0	44	43	Serial or Serial0 (*)
	UART1	15	16	Serial1
	UART2	19	20	Serial2
ESP32-C3	UART0	20	21	Serial or Serial0 (*)

MCU	UART	RX GPIO	TX GPIO	Note
	UART1	18	19	Serial1

El Arduino Mega 2560 cuenta con 4 puertos serie UART por hardware (puertos serie TTL a 5V). Estos UART permiten la comunicación serie simultánea con múltiples dispositivos, además de la conexión USB para la programación, distribuidos en los siguientes pines:

- UART 0: Pines 0 (RX) y 1 (TX) - Usado también para USB.
- UART 1: Pines 19 (RX) y 18 (TX).
- UART 2: Pines 17 (RX) y 16 (TX).
- UART 3: Pines 15 (RX) y 14 (TX).

Página intencionadamente en blanco

Desarrollo de la práctica:

Primera parte: Crear un programa que envíe datos a través de uno de los puertos COM.

Para realizar la práctica, decidimos crear 2 versiones del programa que envíe datos. En la primera, creamos una versión para ordenador (escrita en python), en donde intentábamos abrir el puerto con el baudrate definidos y una vez abierto, simplemente hacíamos un `serial.write` de nuestro mensaje.

Versión para ordenador

A screenshot of a code editor with a dark background and light-colored text. The code is a Python script for sending data over a serial port. It includes imports for 'serial' and 'time', defines 'PORT' as 'COM7' and 'BAUDRATE' as '9600', and a 'main' function that attempts to open the serial port, sends the character 'A', and prints a confirmation message. Error handling is included for 'serial.SerialException'. The script is executed as the main module.

```
import serial
import time

PORT = "COM7"
BAUDRATE = 9600

def main():
    try:
        with serial.Serial(PORT, BAUDRATE, timeout=1) as ser:
            time.sleep(2) # Esperar a que el dispositivo se estabilice
            while True:
                mensaje = "A"
                ser.write(mensaje.encode("utf-8"))

                print("Mensaje enviado correctamente")
                time.sleep(1);

    except serial.SerialException as e:
        print(f"Error abriendo el puerto: {e}")

if __name__ == "__main__":
    main()
```

En la segunda versión, hicimos lo mismo pero para la ESP32, en donde definimos 2 conexiones serial. Siendo Serial "a secas" la conexión por el cable usb serial al ordenador para poder hacer debug y de esa manera saber que las cosas se estaban enviando bien, y Serial2 la conexión de la ESP32 con sus puertos TX y RX al cable USB TTL del ordenador. En la captura aparece también la lectura de datos pero esto es debido a que usamos esa parte del programa para comprobar que la versión para el ordenador funcionaba.

```
// ESP32 UART2
#define RXD2 17
#define TXD2 18

void setup() {
    Serial.begin(600);
    Serial2.begin(600, SERIAL_8N1, RXD2, TXD2);

    Serial.println("ESP32 listo");
}

void loop() {
    // Enviar datos
    Serial2.print("A");
    Serial.println("A");

    // Leer si hay datos
    /*if (Serial2.available()) {
        String data = Serial2.readStringUntil('\n');
        Serial.print("Recibido: ");
        Serial.println(data);
    }*/

    delay(100);
}
```

Conectamos la sonda del osciloscopio al pin Tx de la ESP32 o del cable USB a TTL dependiendo de si es el ordenador el que envía datos o si es la ESP32 la que los está enviando. La conexión al pin Tx se puede ver en la siguiente foto:

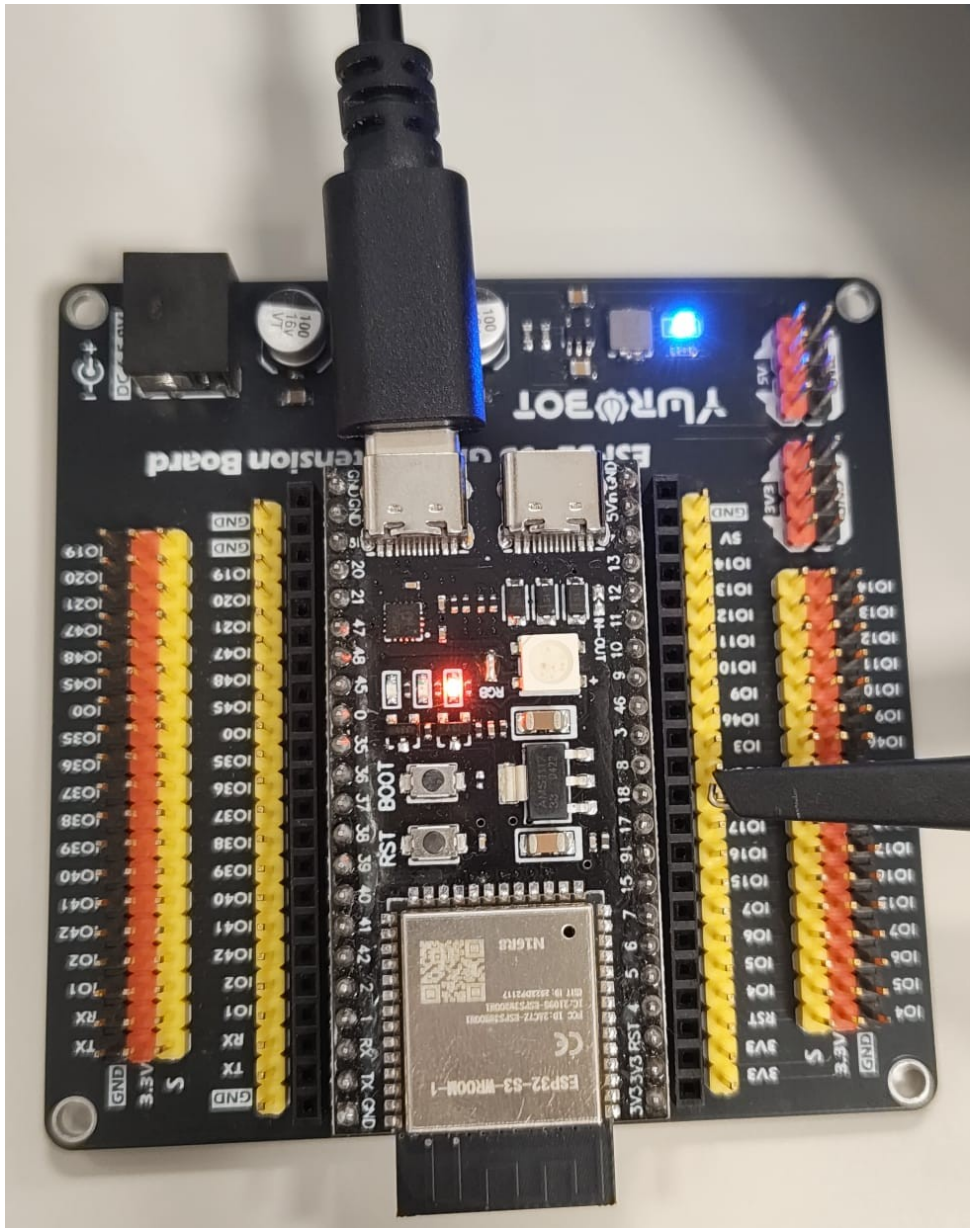


Figura 3: Conexión de la sonda del osciloscopio al pin 18 de la ESP32

Originalmente nos salía la representación del carácter A seguido de algunas cosas que no entendíamos, pero acabamos descubriendo que era debido a que imprimíamos un `\n` al acabar, por lo que lo acabamos eliminando y el resultado (ajustado para que quede algo centrado y visiblemente bonito) es el siguiente:

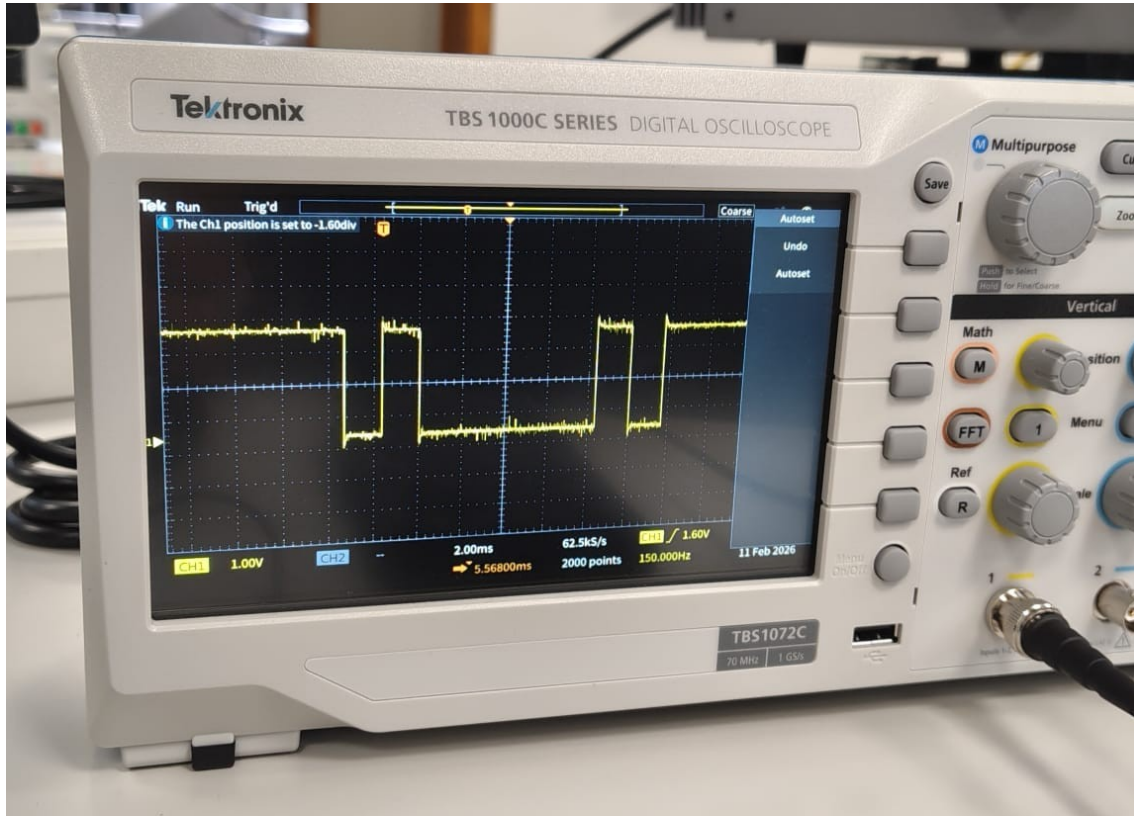


Figura 4: Visualización de la letra A enviada por UART en el osciloscopio

Si analizamos la señal, podemos ver que tenemos la primera bajada corresponde al bit de inicio y le indica al dispositivo receptor que se van a empezar a enviar datos, seguida de los bits que representan la A en binario (osea 0b01000001) pero vista de atrás para adelante y seguida del bit de fin, que (al ser un bit a 1) se mezcla con el estado “normal” de la señal. Cada bit ocupa en la foto aproximadamente un 85% de los cuadrados que salen en el osciloscopio, por lo que la bajada larga ocupa 5 bits aunque parezca que ocupe 4 cuadrados y un trozo.

Ampliaciones

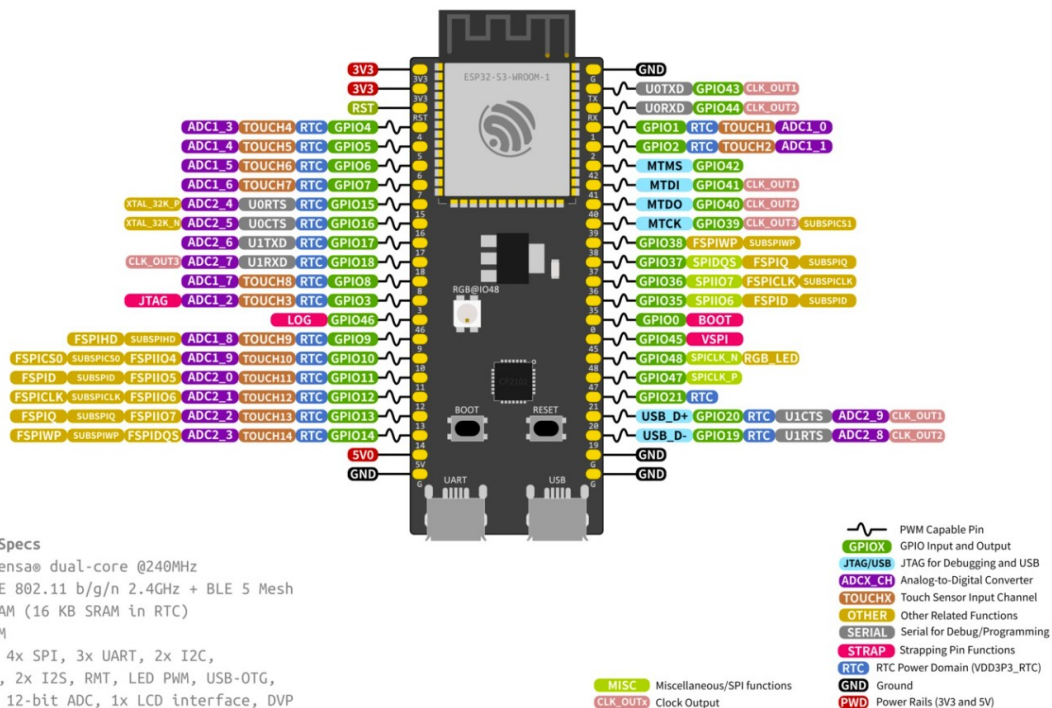
1. Transmitir un fichero entre dos ordenadores.



Debido a que en el laboratorio nuestro equipo solo tenía un cable USB TTL y una ESP32, esta ampliación no se pudo hacer.

2. Transmitir un fichero entre un ordenador y una placa de desarrollo (ESP32-S3-DevKitC-1)

ESP32-S3-DevKitC-1



En la placa de desarrollo con la que se cuenta en el laboratorio (ESP32-S3-DevKitC-1) se aconseja utilizar la UART 1 (GPIO 17: U1TX, GPIO18:U1RX, GPIO19: U1RTS y GPIO20: U1CTS)

Debido a que no vimos cómo es que una ESP32 sin almacenamiento externo pudiera almacenar un fichero, asumimos que lo que se tenía que hacer es simplemente el envío de datos por parte de la ESP32 al ordenador y viceversa, para lo cual usamos el siguiente código a parte de los otros 2 que ya se han mostrado anteriormente:

```
import serial

PUERTO = "COM7"
BAUDRATE = 9600

def main():
    try:
        with serial.Serial(PUERTO, BAUDRATE, timeout=1) as ser:
            print(f"Escuchando en {PUERTO} a {BAUDRATE} baudios...")

            while True:
                if ser.in_waiting > 0:
                    caracter = ser.read().decode("utf-8", errors="ignore").strip()
                    if caracter:
                        print("Recibido:", caracter)

    except serial.SerialException as e:
        print("Error abriendo el puerto:", e)

    except KeyboardInterrupt:
        print("\nPrograma terminado por el usuario.")

if __name__ == "__main__":
    main()
```

Como no le sacamos una foto al montaje usado, se ha recreado digitalmente y el montaje es el siguiente:

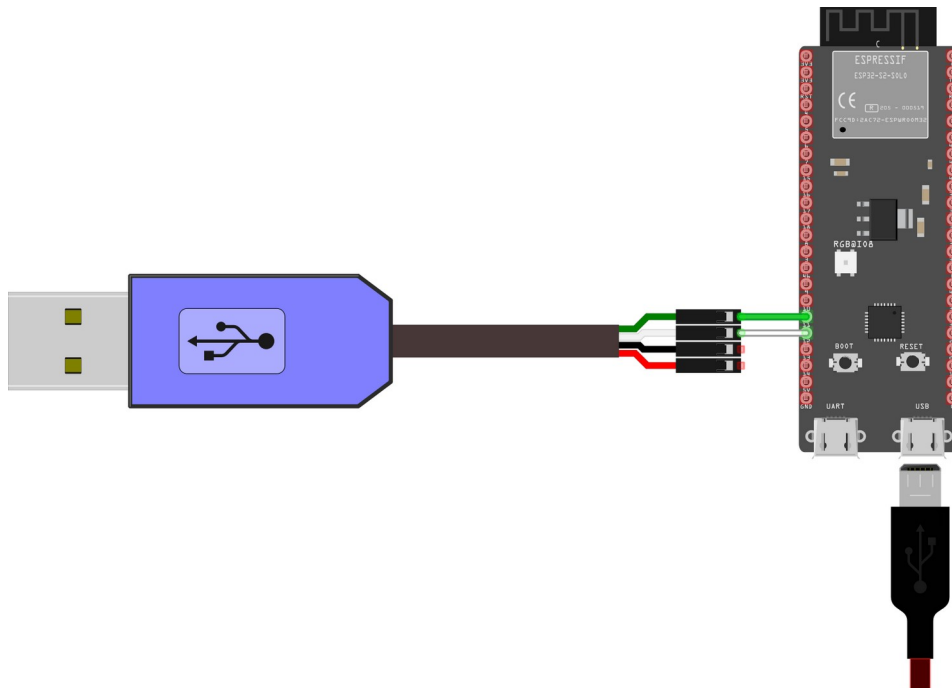
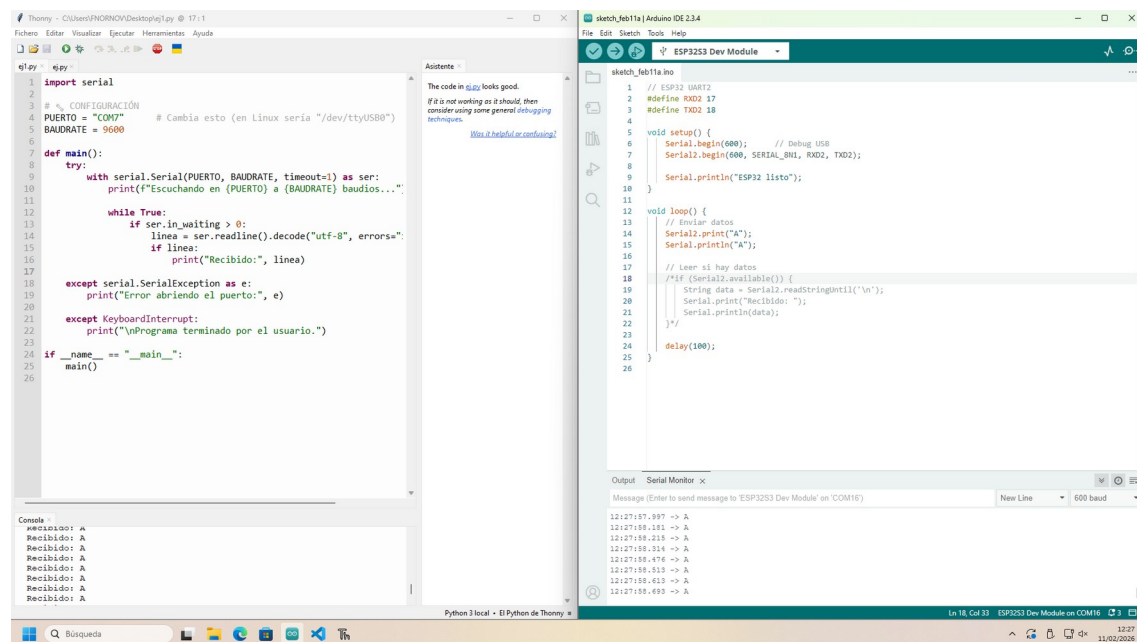


Figura 5: Conexión de la ESP32 mediante UART al cable USB TTL del ordenador y al ordenador mediante USB

No se conectó ni el cable de 5V ni el de GND debido a que la ESP32 ya recibía potencia desde el cable USB. Por lo demás, se conectó el cable verde (correspondiente con la señal tx del cable USB TTL) al pin 17 (rx en la ESP32) y el pin 18 (tx en la ESP32) al cable blanco (rx en el cable USB TTL).

Durante la realización de esta parte, llegamos a un problema que básicamente consistía en que originalmente le habíamos colocado a la lectura de datos una espera y por consiguiente, a veces cuando se leían los datos, en vez de leerse una A a la vez, se leían varias ya que las As se había quedado esperando en el buffer Rx. Al eliminar la espera, el acceso al buffer se volvió casi instantáneo y con eso pudimos solventar el error.

El resultado del envío de datos es el siguiente:



```
1 import serial
2
3 # CONFIGURACIÓN
4 PUERTO = "COM7" # Cambia esto (en Linux sería "/dev/ttyUSB0")
5 BAUDRATE = 9600
6
7 def main():
8     try:
9         with serial.Serial(PUERTO, BAUDRATE, timeout=1) as ser:
10             print(f"Escuchando en {PUERTO} a {BAUDRATE} baudios...")
11
12             while True:
13                 if ser.in_waiting > 0:
14                     linea = ser.readline().decode("utf-8", errors="")
15                     if linea:
16                         print("Recibido:", linea)
17
18             except serial.SerialException as e:
19                 print("Error abriendo el puerto:", e)
20
21             except KeyboardInterrupt:
22                 print("\nPrograma terminado por el usuario.")
23
24 if __name__ == "__main__":
25     main()
26
```

```
1 // ESP32 UART2
2 #define RXD2 17
3 #define TXD2 18
4
5 void setup() {
6     Serial.begin(600); // Debug USB
7     Serial2.begin(600, SERIAL_BN1, RXD2, TXD2);
8     Serial.println("ESP32 listo");
9 }
10
11 void loop() {
12     // Enviar datos
13     Serial2.print("A");
14     Serial.println("A");
15
16     // Leer si hay datos
17     if (Serial2.available()) {
18         String data = Serial2.readStringUntil('\n');
19         Serial.print("Recibido: ");
20         Serial.println(data);
21     }
22     delay(100);
23 }
24
25
```

Output Serial Monitor x

Message (Enter to send message to 'ESP32S3 Dev Module' on 'COM16')

New Line 600 baud

12:27:57.997 -> A
12:27:58.181 -> A
12:27:58.218 -> A
12:27:58.216 -> A
12:27:58.476 -> A
12:27:58.513 -> A
12:27:58.613 -> A
12:27:58.693 -> A

Figura 6: Comunicación ESP32 <-> Ordenador

3. Transmitir un fichero entre dos placas de desarrollo (ESP32-S3-DevKitC-1)

Esta ampliación no pudimos realizarla debido a la falta de tiempo y a que justo el kit que usó nuestro equipo no poseía una segunda ESP32.

Apéndice A.

El control de flujo por hardware UART es una técnica que permite que dispositivos con diferentes velocidades de procesamiento se comuniquen de forma fiable a través de una conexión UART sin pérdida de datos.

Por ejemplo, cuando el dispositivo A transmite un flujo continuo de bytes al dispositivo B, y este funciona a menor velocidad que el dispositivo A, el dispositivo B puede indicar al dispositivo A que pause temporalmente la transmisión, lo que le da tiempo para procesar los datos ya recibidos y liberar búferes de recepción. El control de flujo lo hace posible proporcionando señales adicionales que indican al dispositivo A que pause o reanude la transmisión de datos según sea necesario. Estas señales adicionales se denominan RTS (solicitud de envío) y CTS (autorización de envío).

Estas señales están interconectadas entre los dos dispositivos, donde el RTS de un dispositivo se conecta al CTS del otro, y viceversa. Cada dispositivo escribe su RTS para indicar que está listo para recibir nuevos datos, mientras que lee el CTS para determinar si puede enviar datos al dispositivo homólogo. Un dispositivo mantiene la línea RTS activada mientras esté preparado para aceptar datos adicionales y la desactiva poco antes de que su búfer de recepción alcance su capacidad máxima. El control de flujo por hardware funciona en ambas direcciones, lo que significa que cada dispositivo (Dispositivo A o Dispositivo B) puede indicar una pausa en la transmisión.

El periférico UART de los SoC Nordic admite el control automático de flujo por hardware, que se describe en detalle en la hoja de datos (especificaciones del producto) de cada SoC. En Periféricos -> UART -> Transmisión y Recepción.

Referencias Bibliográficas:

<https://academy.nordicsemi.com/courses/nrf-connect-sdk-fundamentals/lessons/lesson-4-serial-communication-uart/topic/uart-protocol/>

Short introduction—pySerial 3.5 documentation. (s. f.). Recuperado 14 de febrero de 2026, de <https://pyserial.readthedocs.io/en/latest/shortintro.html>

Serial (UART)——Arduino ESP32 latest documentation. (s. f.). Recuperado 14 de febrero de 2026, de <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/serial.html>